

# markdown-to-web — Validation Review & Generalized Skill Specification

**Document version:** 1.0  
**Date:** 2026-08-06  
**Scope:** (1) Audit of `react-markdown-report` v1.0.1 skill; (2) design of a generalized replacement, `markdown-to-web` v2.0.0  
**Reviewer:** Super Z (GLM)  
**Verification protocol:** Desk review. Findings tagged Verified / Reasoned / Assumed per the skill's own evidence contract. No code execution in this environment — recommendations that depend on runtime behavior are explicitly marked.

## Part 1 — Validation Review of `react-markdown-report` v1.0.1

### 1.0 Executive Summary

The skill is a well-organized, single-purpose project skill for a React 19 + Vite 7 + Tailwind v4 single-file web rendering of one specific Markdown audit report. Its strengths are an explicit evidence contract, a code-first design system, and a refreshingly anti-generic visual mandate. Its weaknesses are an over-fit scope (one report, one design), several accessibility gaps that contradict its own WCAG AAA claim, no automated quality gate beyond `tsc --noEmit && npm run build`, and runtime font dependence that breaks the "single-file portability" promise.

**Severity counts (findings detailed in §1.2):**

| Severity      | Count | Examples                                                                                                                                                     |
|---------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Critical      | 0     | —                                                                                                                                                            |
| High          | 3     | WCAG AAA over-claim; no <code>prefers-reduced-motion</code> ; fonts not inlined despite "single-file portability" promise                                    |
| Medium        | 7     | No automated a11y CI; touch targets < 44 px; fixed badge keys; slug parity unverified; no print CSS; no theme parameterization; dead <code>cn.ts</code> code |
| Low           | 5     | No <code>md / xL</code> breakpoints; single template; no i18n hooks; no search; no <code>prefers-color-scheme</code>                                         |
| Informational | 4     | No CI, no tests, no lint, stale Appendix A                                                                                                                   |

**Overall verdict:** The skill is internally consistent and high-quality for its narrow purpose. It is **not reusable** as-is for a different Markdown document without forking. The right path forward is to extract its durable patterns (evidence contract, code-first theming, slug-parity discipline, single-file build) into

a generalized skill — `markdown-to-web` v2.0.0 in Part 2 — while fixing the accessibility and portability gaps identified here.

## 1.1 Methodology

Each finding below follows the format mandated by the skill's own Section 12:

- **Location** (section reference in the v1.0.1 skill document)
- **Description**
- **Evidence** (quoted or paraphrased from the skill)
- **Impact**
- **Severity** (Critical / High / Medium / Low / Informational)
- **Confidence** (Verified / Reasoned / Assumed — see note below)
- **Recommended fix**

**Confidence note:** Because the skill was reviewed as text only (no project bootstrap, no `npm install`, no `axe` run, no Lighthouse pass), most findings are **Reasoned** (logical inference from the skill's own statements) or **Assumed** (inference about runtime behavior the skill does not measure). Where the skill itself contradicts its own claims (e.g., "WCAG AAA" vs. documented 36×36 px touch targets), the finding is **Verified** — the contradiction is in the text.

## 1.2 Section-by-Section Findings

Findings are ordered by severity within each section, then by section number.

---

### §1 PROJECT IDENTITY & DESIGN PHILOSOPHY

#### Finding 1.1 — Scope is hardcoded to one report

- **Location:** §1, "One-sentence description"
- **Description:** The skill's identity sentence fixes it to "a structural/content/IA comparative audit of `kelp.agency` vs. its clone `astro.jesspete.shop`." Every downstream module (badge keys, content path `src/content/comparative-analysis.md`, hero copy in `App.tsx:124`) inherits that fixation.
- **Evidence:** "A single-file, zero-backend React application that renders a structural/content/IA comparative audit of `kelp.agency` vs. its clone `astro.jesspete.shop`."
- **Impact:** The skill cannot be invoked for any other Markdown document without forking. An agent encountering "render this README as a polished web page" will not match this skill's trigger surface.
- **Severity:** High
- **Confidence:** Verified (textually explicit)
- **Recommended fix:** Generalize the identity to "renders an arbitrary Markdown document as a polished single-file web page using a configurable template and evidence-tag protocol." Move the audit-report specifics to an example in the appendix.

#### Finding 1.2 — "No generic UI" mandate conflicts with reuse

- **Location:** §1, "Anti-generic mandate (explicitly rejected)"
- **Description:** The skill explicitly rejects "any component that could be dropped into a different

project without visual friction." This is a legitimate aesthetic position for a one-off report but is incompatible with a generalized skill, which by definition must serve multiple documents and audiences.

- **Evidence:** The skill lists purple gradients, predictable card grids, and Inter/Roboto neutrality as "explicitly rejected."
  - **Impact:** Future agents generalizing this skill may feel constrained to preserve the bespoke editorial palette even when a different document (e.g., a technical API reference) calls for a different visual register.
  - **Severity:** Medium
  - **Confidence:** Reasoned
  - **Recommended fix:** Reframe the mandate as "default to intentional, editorial design; allow template-level override." Provide multiple templates (editorial / technical / minimal) so the anti-generic ethos is preserved per-template, not hard-coded.
- 

## §2 TECH STACK & ENVIRONMENT

### Finding 2.1 — Versions are pinned and verified

- **Location:** §2, tech stack table
- **Description:** Every dependency is pinned to an exact version (React 19.2.6, Vite 7.3.2, Tailwind 4.1.17, react-markdown 10.1.0, etc.) and the skill cross-references `package.json`.
- **Evidence:** "`cat package.json` → every row above matches `dependencies` / `devDependencies` exactly."
- **Impact:** Reproducibility is high; an agent rebuilding from this skill will not face version drift.
- **Severity:** Informational (positive finding)
- **Confidence:** Reasoned (not re-verified by reading `package.json`)
- **Recommended fix:** Carry this discipline forward into v2.0.0. Add a `npm ls --depth=0` command to the pre-ship checklist as a verification gate.

### Finding 2.2 — `github-slugger` and `rehype-slug` parity is asserted, not verified

- **Location:** §2, "TOC extraction" row; §7.3
- **Description:** The skill states `github-slugger` 2.0.0 "must stay compatible with `rehype-slug`'s output." It does not provide a test or runtime assertion that the two remain in sync.
- **Evidence:** "Slugs generated by `github-slugger` — **must match** `rehype-slug` output (both use same algorithm) or anchor links break."
- **Impact:** A future patch upgrade to either package could silently break anchor navigation. The skill's own §9 lists "Anchor link mismatch" as anti-pattern #3, acknowledging the risk.
- **Severity:** Medium
- **Confidence:** Reasoned
- **Recommended fix:** Add a unit test that runs both slugs over a fixture of headings (CJK, emoji, code, repeated headings, leading/trailing whitespace) and asserts equality. Run it in pre-commit.

### Finding 2.3 — Node version floor is correct for Vite 7

- **Location:** §2, "Node" row
- **Description:** The skill requires Node `≥20.19` or `≥22.12`, matching Vite 7's official requirement.

- **Location:** §2, "Node" row
  - **Description:** The skill requires Node `≥20.19` or `≥22.12`, matching Vite 7's official requirement.
  - **Evidence:** Stated explicitly.
  - **Impact:** Prevents the most common Vite 7 bootstrap failure (older Node).
  - **Severity:** Informational (positive finding)
  - **Confidence:** Reasoned
  - **Recommended fix:** None. Carry forward into v2.0.0.
- 

### §3 BOOTSTRAPPING & CONFIGURATION

#### Finding 3.1 — No `tsc` npm script

- **Location:** §3.1, "Typecheck (no npm script exists — run directly)"
- **Description:** Typechecking is documented as `npx tsc --noEmit` because no `npm run typecheck` script exists.
- **Evidence:** Explicit in the commands block.
- **Impact:** Agents may forget to typecheck before building; the pre-ship checklist depends on remembering the `npx` invocation.
- **Severity:** Low
- **Confidence:** Verified (textually explicit)
- **Recommended fix:** Add `"typecheck": "tsc --noEmit"` to `package.json` scripts. Pre-ship becomes `npm run typecheck && npm run build`.

#### Finding 3.2 — Google Fonts `@import` requires runtime network

- **Location:** §3.3, "Google Fonts loaded via `@import` in CSS — requires network at runtime; single-file build does **not** inline fonts"
  - **Description:** The skill correctly documents that `vite-plugin-singlefile` inlines JS/CSS but not `@import` ed fonts. It does not provide an offline alternative.
  - **Evidence:** Stated explicitly; reinforced in §12 lesson 4 and §13 pitfall "Assume `dist/index.html` works offline."
  - **Impact:** The "single-file portability" promise in §1 is partially false — the artifact depends on a CDN. In air-gapped, offline, or archival contexts, fonts fall back to system serif/sans/mono, breaking the bespoke editorial design.
  - **Severity:** High
  - **Confidence:** Verified (textually explicit; the skill both claims portability and admits the font gap)
  - **Recommended fix:** In v2.0.0, offer an `--offline` build mode that downloads the Google Fonts subset at build time (via `@fontsource` packages or `vite-plugin-fonts`) and inlines the font files as base64 data URIs. Default to CDN; document the tradeoff.
- 

### §4 THE DESIGN SYSTEM (CODE-FIRST)

#### Finding 4.1 — `@theme` tokens are well-structured

- **Location:** §4.1
- **Description:** The skill defines a coherent token set (ink/paper/teal/moss scale + 5 severity tokens) using Tailwind v4's `@theme` directive, with no `tailwind.config.js`.

- **Location:** §4.1
- **Description:** The skill defines a coherent token set (ink/paper/teal/moss scale + 5 severity tokens) using Tailwind v4's `@theme` directive, with no `tailwind.config.js`.
- **Evidence:** Code block at §4.1 lines 848–873.
- **Impact:** Token-to-class derivation is clean; adding a new color is a one-line change.
- **Severity:** Informational (positive finding)
- **Confidence:** Reasoned
- **Recommended fix:** Carry forward; generalize by making the token set a template-level concern (each template ships its own `@theme`).

#### Finding 4.2 — Severity palette is hardcoded to audit-report semantics

- **Location:** §4.1, `--color-critical / high / medium / low / info` tokens
- **Description:** Five severity tokens bake in the audit-report vocabulary. A different document (e.g., a changelog with `added / changed / deprecated / removed`) cannot reuse the palette without adding tokens.
- **Evidence:** Token names are literal `critical`, `high`, etc.
- **Impact:** Limits reuse.
- **Severity:** Medium
- **Confidence:** Verified
- **Recommended fix:** In v2.0.0, replace fixed semantic tokens with a generic 5-step accent scale (`accent-1` through `accent-5`) plus a "kind" registry that maps document-specific tags (`critical`, `added`, `breaking`, etc.) to accent steps. Templates provide a default kind-to-step mapping; documents can override.

#### Finding 4.3 — No `prefers-reduced-motion` guard

- **Location:** §4.5, `html { scroll-behavior: smooth; } ; §8` acknowledges this as a gap.
- **Description:** `scroll-behavior: smooth` is set globally without a `prefers-reduced-motion: reduce` override. The skill flags this as a known gap but does not fix it.
- **Evidence:** "Not implemented — `html { scroll-behavior: smooth; }` does NOT auto-respect `prefers-reduced-motion`; add `@media (prefers-reduced-motion: reduce) { html { scroll-behavior: auto; } }` to enable."
- **Impact:** Users with vestibular disorders get unwanted smooth-scroll animation on TOC clicks. This is a WCAG 2.3.3 (AAA) failure and a 2.2.2 concern.
- **Severity:** High
- **Confidence:** Verified (the skill both implements the unguarded behavior and documents the missing guard)
- **Recommended fix:** Add the media query block the skill itself recommends. One-line CSS fix.

#### Finding 4.4 — No `prefers-color-scheme: dark` support

- **Location:** §4 (entire)
- **Description:** The design system is light-only. There is no dark theme token set or media query.
- **Evidence:** Absence — no `--color-*` dark variants, no `@media (prefers-color-scheme: dark)` anywhere in §4 or §8.
- **Impact:** Users who prefer dark mode get a bright paper-50 page; in low-light reading contexts this is a usability regression. Not a WCAG failure, but a polish gap.

- **Location:** §4 (entire)
  - **Description:** The design system is light-only. There is no dark theme token set or media query.
  - **Evidence:** Absence — no `--color-*` dark variants, no `@media (prefers-color-scheme: dark)` anywhere in §4 or §8.
  - **Impact:** Users who prefer dark mode get a bright paper-50 page; in low-light reading contexts this is a usability regression. Not a WCAG failure, but a polish gap.
  - **Severity:** Low
  - **Confidence:** Reasoned
  - **Recommended fix:** In v2.0.0, make the design system dual-mode by default. Each template defines both light and dark token sets; `@media (prefers-color-scheme: dark)` swaps them. Add a manual toggle in the header for explicit override.
- 

## §5 COMPONENT ARCHITECTURE & PATTERNS

### Finding 5.1 — `cn` utility is dead code

- **Location:** §5.1, file inventory row "`src/utils/cn.ts ... (currently unused)`"; §9 anti-pattern #4; §13 pitfall
- **Description:** The `cn()` helper (`clsx + tailwind-merge`) is imported nowhere in the render path. The skill documents this in three places.
- **Evidence:** "`cn.ts` currently dead code."
- **Impact:** Minor — strict `tsc` catches it; no runtime effect. But it suggests the codebase was scaffolded with `shadcn/ui` conventions in mind and then simplified, leaving scaffolding behind.
- **Severity:** Low
- **Confidence:** Verified (skill is explicit)
- **Recommended fix:** Either delete `cn.ts` (preferred — YAGNI) or actually use it in `MarkdownReport.tsx` for conditional class composition when adding template/theme variants.

### Finding 5.2 — `enhanceReportMarkdown` runs at render time

- **Location:** §5.3, "Renderer as Configuration"
- **Description:** The regex preprocessor is called inside `MarkdownReport`'s render path. The skill notes this is "pure, cheap."
- **Evidence:** "`enhanceReportMarkdown()` called at render time (pure, cheap)."
- **Impact:** For a 244-line report, negligible. For a 10,000-line document (the v2.0.0 use case), running a global regex on every re-render is wasteful.
- **Severity:** Low
- **Confidence:** Reasoned
- **Recommended fix:** In v2.0.0, memoize the enhancement via `useMemo` keyed on the markdown string, or run it once at module load (since the markdown is a static `?raw` import).

### Finding 5.3 — Single state ( `drawerOpen` ) is correct for this scope

- **Location:** §5.3, "`App.tsx` — Single state, composed layout"
- **Description:** The skill correctly limits client state to a single boolean. This is good React hygiene.
- **Evidence:** "Only state is `drawerOpen` in `App.tsx` ; keep it that way."
- **Impact:** Positive — easy to reason about, no state management library needed.



- **Location:** §5.3, " `App.tsx` — Single state, composed layout"
  - **Description:** The skill correctly limits client state to a single boolean. This is good React hygiene.
  - **Evidence:** "Only state is `drawerOpen` in `App.tsx` ; keep it that way."
  - **Impact:** Positive — easy to reason about, no state management library needed.
  - **Severity:** Informational (positive finding)
  - **Confidence:** Reasoned
  - **Recommended fix:** Preserve this discipline in v2.0.0. Add `activeSection` (for TOC highlight) and `theme` (for manual dark/light toggle) as the only additional state — both derivable from URL hash or `localStorage` .
- 

## §6 CUSTOM HOOKS DEEP DIVE

### Finding 6.1 — Explicit "None exist" is excellent documentation

- **Location:** §6
  - **Description:** The skill explicitly states no custom hooks exist, preventing future agents from searching for a `hooks/` directory.
  - **Evidence:** "Documented here explicitly so future agents don't search for a `hooks/` directory."
  - **Impact:** Saves onboarding time; sets a clear expectation.
  - **Severity:** Informational (positive finding)
  - **Confidence:** Verified
  - **Recommended fix:** Carry this pattern forward — every v2.0.0 section that could be empty (e.g., "Custom Server Logic") should explicitly say "None."
- 

## §7 CONTENT MANAGEMENT & DATA INGESTION

### Finding 7.1 — Badge protocol is too narrow

- **Location:** §7.2
- **Description:** The badge system recognizes exactly 9 keys: 5 severity ( `critical` / `high` / `medium` / `low` / `informational` ) + 4 confidence ( `verified` / `reasoned` / `assumed` / `unverifiable` ). The regex in `enhance.ts` only matches `**Severity:**` or `**Confidence:**` bullets.
- **Evidence:** Badge keys table at §7.2.
- **Impact:** A changelog ( `Added` / `Changed` / `Fixed` ), a status report ( `Done` / `In Progress` / `Blocked` ), or a compliance matrix ( `Pass` / `Fail` / `N/A` ) cannot use the badge system without code changes to `enhance.ts` and `StatusBadge` .
- **Severity:** Medium
- **Confidence:** Verified
- **Recommended fix:** Generalize the protocol. In v2.0.0, the regex matches any `**<Tag>:** <value>` bullet where `<Tag>` is registered in a `TAG_REGISTRY` (a JSON or TS module). Templates ship default registries; documents can extend. See Part 2 §8.

### Finding 7.2 — `enhance.ts` regex is fragile to formatting variation

- **Location:** §7.2; §15 pattern
- **Description:** The regex `/^\(s*-s*\*(?:Severity|Confidence):*\*)s+(.)$/gm` requires the

bullet to start with `- **Severity:**` or `- **Confidence:**` exactly. Variations like `* **Severity:**` (asterisk bullet), `1. **Severity:**` (ordered list), or `**Severity:** critical` (no bullet) are silently skipped.

- **Evidence:** Regex quoted in §15.
- **Impact:** Authors writing Markdown naturally use varied bullet styles; silent skip means badges don't render and the author has no feedback.
- **Severity:** Medium
- **Confidence:** Reasoned
- **Recommended fix:** Either (a) accept all bullet styles ( `[-*+]` and ordered `\d+` ), or (b) emit a build-time warning when a line contains `**<Tag>:**` but doesn't match the full pattern. Option (b) is more defensive.

### Finding 7.3 — TOC contract correctly nests H3 under H2

- **Location:** §7.3
- **Description:** `buildToc()` extracts only H2/H3, nests H3 under the most recent H2, and handles orphan H3s by promoting them to top-level.
- **Evidence:** "Orphan H3s (no preceding H2) become top-level."
- **Impact:** Positive — TOC is predictable and matches reader expectations.
- **Severity:** Informational (positive finding)
- **Confidence:** Reasoned
- **Recommended fix:** In v2.0.0, extend to H4 (configurable depth) since technical-docs templates often have 4-level hierarchies.

---

## §8 ACCESSIBILITY (WCAG AAA) IMPLEMENTATION

### Finding 8.1 — "WCAG AAA" claim is partially false

- **Location:** §1 "WCAG AAA where feasible"; §8 touch targets row
- **Description:** The skill claims "WCAG AAA where feasible" but §8 documents that drawer buttons are 36×36 px and 32×32 px, failing WCAG 2.5.5 (AAA, 44×44 px). The skill honestly notes "fail 2.5.5 AAA (44px)."
- **Evidence:** "Drawer buttons: menu 36×36px ( `p-2` + 20px icon), close 32×32px ( `p-1.5` ) — pass WCAG 2.5.8 min (24px), **fail 2.5.5 AAA (44px).**"
- **Impact:** The headline claim ("WCAG AAA") overstates the actual conformance. A consumer reading only §1 will believe AAA is met.
- **Severity:** High
- **Confidence:** Verified (internal contradiction in the skill text)
- **Recommended fix:** Either (a) increase touch target sizes to 44×44 px (preferred — use `p-2.5` or larger), or (b) restate the claim as "WCAG 2.2 AA; AAA-targeted where feasible, with documented exceptions in §8." Option (a) is the right v2.0.0 default.

### Finding 8.2 — Focus styles rely on browser default

- **Location:** §8, "Focus visible" row
- **Description:** Only the skip link has explicit `focus:` classes. Other interactive elements (TOC links, nav links, badges) rely on the browser default outline.



- **Location:** §8, "Focus visible" row
- **Description:** Only the skip link has explicit `focus:` classes. Other interactive elements (TOC links, nav links, badges) rely on the browser default outline.
- **Evidence:** "Other interactive elements rely on the browser default focus outline (no `focus:` classes present)."
- **Impact:** Browser default outlines are inconsistent (Safari's blue ring, Chrome's black ring, Firefox's dotted). Some default outlines fail 1.4.11 (Non-text Contrast, 3:1).
- **Severity:** Medium
- **Confidence:** Reasoned
- **Recommended fix:** Add a global `:focus-visible` style in `index.css`: `outline: 2px solid var(--color-teal-600); outline-offset: 2px; .` Apply consistently to all interactive elements.

### Finding 8.3 — No automated a11y test in pre-ship

- **Location:** §11 Pre-Ship Checklist
- **Description:** The quality gate is `tsc --noEmit && npm run build && npm run preview` with manual smoke checks. No `axe`, `Lighthouse`, or `pa11y` run.
- **Evidence:** "No other gates exist. No lint, no test suite, no CI."
- **Impact:** Accessibility regressions can ship undetected. The skill's §8 "Gaps" note acknowledges this: "No automated axe/Lighthouse run in CI."
- **Severity:** Medium
- **Confidence:** Verified
- **Recommended fix:** In v2.0.0, add `@axe-core/playwright` or `lighthouse` to devDependencies. Pre-ship command becomes `npm run typecheck && npm run a11y && npm run build`. Fail the build on critical/serious a11y violations.

### Finding 8.4 — Badge text contrast fails AAA

- **Location:** §8, "Color contrast" row
- **Description:** The skill computes badge text pairs at 4.76–6.99:1, passing AA but failing AAA for 12 px normal text.
- **Evidence:** "Badge text pairs = **4.76–6.99:1 (AA ✓, AAA ✗)** — badges are 12px normal text."
- **Impact:** Low-vision users may struggle with badge text. Since badges carry semantic meaning (severity, confidence), this is content, not decoration.
- **Severity:** Medium
- **Confidence:** Verified (skill self-reports)
- **Recommended fix:** Either increase badge text size to 14 px (which relaxes AAA threshold to 4.5:1) or darken the badge text colors until all pairs clear 7:1.

---

## §9 ANTI-PATTERNS & COMMON BUGS

### Finding 9.1 — Anti-pattern table is high-value

- **Location:** §9
- **Description:** The five-row anti-pattern table maps symptom → root cause → fix with concrete file references. This is exactly the format future agents need.
- **Evidence:** Table at §9 lines 1040–1047.

- **Location:** §9
  - **Description:** The five-row anti-pattern table maps symptom → root cause → fix with concrete file references. This is exactly the format future agents need.
  - **Evidence:** Table at §9 lines 1040–1047.
  - **Impact:** Positive — significantly reduces debugging time.
  - **Severity:** Informational (positive finding)
  - **Confidence:** Verified
  - **Recommended fix:** Carry forward and expand. In v2.0.0, each anti-pattern should link to a fixture or unit test that reproduces it.
- 

## §10 DEBUGGING GUIDE

### Finding 10.1 — Debugging guide is symptom-cause-fix structured

- **Location:** §10
  - **Description:** Six-row table mapping common symptoms to causes and fixes.
  - **Evidence:** Table at §10 lines 1052–1059.
  - **Impact:** Positive.
  - **Severity:** Informational (positive finding)
  - **Confidence:** Reasoned
  - **Recommended fix:** Add a "reproducible test case" column in v2.0.0 so each row links to a minimal reproduction.
- 

## §11 PRE-SHIP CHECKLIST

### Finding 11.1 — Quality gate is too narrow

- **Location:** §11
  - **Description:** Only two automated commands ( `tsc --noEmit` , `npm run build` ) plus a manual smoke test. No lint, no a11y, no unit tests, no format check.
  - **Evidence:** "No other gates exist. No lint, no test suite, no CI."
  - **Impact:** The skill itself acknowledges this as a limitation. For a generalized skill targeting multiple document types and authors, this gate is insufficient.
  - **Severity:** High (elevated from Medium because v2.0.0 has broader scope)
  - **Confidence:** Verified
  - **Recommended fix:** v2.0.0 pre-ship: `npm run typecheck && npm run lint && npm run test && npm run a11y && npm run build` . Each is a hard gate.
- 

## §12 LESSONS LEARNT & HOW TO AVOID THEM

### Finding 12.1 — Lessons are well-extracted

- **Location:** §12
- **Description:** Five lessons, each tied to a concrete failure mode and a generalizable principle.
- **Evidence:** Lessons 1–5 at lines 1089–1097.
- **Impact:** Positive — these are the most reusable part of the skill.

- **Location:** §12
  - **Description:** Five lessons, each tied to a concrete failure mode and a generalizable principle.
  - **Evidence:** Lessons 1–5 at lines 1089–1097.
  - **Impact:** Positive — these are the most reusable part of the skill.
  - **Severity:** Informational (positive finding)
  - **Confidence:** Reasoned
  - **Recommended fix:** Carry all five into v2.0.0. Lesson 2 (slug parity) and lesson 4 (single-file ≠ offline fonts) become first-class concerns in v2.0.0.
- 

## §13 PITFALLS TO AVOID

### Finding 13.1 — Pitfalls table is actionable

- **Location:** §13
  - **Description:** Seven rows, each pairing a "Don't" with a "Do."
  - **Evidence:** Table at §13 lines 1103–1111.
  - **Impact:** Positive.
  - **Severity:** Informational (positive finding)
  - **Confidence:** Reasoned
  - **Recommended fix:** Expand in v2.0.0 to cover the new pitfalls introduced by generalization (template selection, tag registry conflicts, font subsetting edge cases).
- 

## §14 BEST PRACTICES

### Finding 14.1 — Best practices are conventional and correct

- **Location:** §14
  - **Description:** Standard React + TypeScript + Tailwind v4 hygiene (functional components, `interface` for shapes, CSS-first theming, `@/*` alias).
  - **Evidence:** Four subsections at lines 1117–1139.
  - **Impact:** Positive — no surprises, easy to onboard.
  - **Severity:** Informational (positive finding)
  - **Confidence:** Reasoned
  - **Recommended fix:** Carry forward unchanged.
- 

## §15 CODING PATTERNS

### Finding 15.1 — Three patterns are documented as code

- **Location:** §15
- **Description:** Three reusable patterns (preprocessor, slugger-sharing, renderer-as-config) with code snippets and "why" rationale.
- **Evidence:** Three pattern blocks at lines 1146–1178.
- **Impact:** Positive — these are the load-bearing abstractions.
- **Severity:** Informational (positive finding)

- **Location:** §15
  - **Description:** Three reusable patterns (preprocessor, slugger-sharing, renderer-as-config) with code snippets and "why" rationale.
  - **Evidence:** Three pattern blocks at lines 1146–1178.
  - **Impact:** Positive — these are the load-bearing abstractions.
  - **Severity:** Informational (positive finding)
  - **Confidence:** Reasoned
  - **Recommended fix:** In v2.0.0, add two more patterns: (a) template composition (how a template overrides default components), (b) tag registry extension (how a document adds custom badges).
- 

## §16 CODING ANTI-PATTERNS

### Finding 16.1 — Anti-patterns table is concrete

- **Location:** §16
  - **Description:** Six rows with "Don't" example, "Correct" example.
  - **Evidence:** Table at lines 1185–1192.
  - **Impact:** Positive.
  - **Severity:** Informational (positive finding)
  - **Confidence:** Reasoned
  - **Recommended fix:** Carry forward; add a row for "hardcoded tag keys" (don't: `if (key === 'critical') ; do: if (TAG_REGISTRY.has(key))` ).
- 

## §17 RESPONSIVE BREAKPOINT REFERENCE

### Finding 17.1 — Only `sm` and `lg` are used

- **Location:** §17
  - **Description:** The codebase uses only `sm`: (640 px) and `lg`: (1024 px) breakpoints. `md` , `xl` , `2xl` are documented as "never used."
  - **Evidence:** "No `md`: / `xl`: / `2xl`: classes anywhere."
  - **Impact:** For a single-column editorial report, two breakpoints suffice. For a technical-docs template with sidebar + code blocks + tables, a `md` (768 px) breakpoint is often needed to handle the sidebar-to-stacked transition separately from the hero-to-condensed transition.
  - **Severity:** Low
  - **Confidence:** Reasoned
  - **Recommended fix:** v2.0.0 templates may use `md` / `xl` where the layout requires it; document the chosen scale per template.
- 

## §18 Z-INDEX LAYER MAP

### Finding 18.1 — Z-index map is explicit and minimal

- **Location:** §18
- **Description:** Three z-index levels ( `z-50` , `z-40` , default) with file:line references.
- **Evidence:** Table at lines 1218–1224.

- **Location:** §18
  - **Description:** Three z-index levels ( `z-50` , `z-40` , default) with file:line references.
  - **Evidence:** Table at lines 1218–1224.
  - **Impact:** Positive — prevents z-index wars.
  - **Severity:** Informational (positive finding)
  - **Confidence:** Verified
  - **Recommended fix:** In v2.0.0, add `z-30` for sticky-in-content elements (e.g., sticky section headers in technical-docs template) and `z-60` for command palette / search overlay.
- 

## §19 COLOR REFERENCE (COMPLETE)

### Finding 19.1 — Color reference is exhaustive and matches `@theme`

- **Location:** §19
  - **Description:** Every token is listed with hex, RGB, Tailwind class, and usage. Badge tint combinations are also listed.
  - **Evidence:** Two tables at lines 1233–1261.
  - **Impact:** Positive — eliminates guesswork.
  - **Severity:** Informational (positive finding)
  - **Confidence:** Reasoned (not re-verified against `index.css` )
  - **Recommended fix:** In v2.0.0, generate this table programmatically from the `@theme` block to prevent drift. A simple script that parses `index.css` and emits the markdown table.
- 

## §20 TYPESCRIPT INTERFACE REFERENCE

### Finding 20.1 — `TocItem` is the only named interface

- **Location:** §20
  - **Description:** All other component props are inline anonymous types. The skill flags this explicitly.
  - **Evidence:** "No named `Props` interfaces exist — all component props are inline anonymous types; `TocItem` ( `toc.ts:3-8` ) is the only named interface in the codebase."
  - **Impact:** Mixed. Inline props are fine for leaf components; for shared abstractions (template config, tag registry), named interfaces improve reuse.
  - **Severity:** Low
  - **Confidence:** Verified
  - **Recommended fix:** In v2.0.0, promote shared types ( `TemplateConfig` , `TagDefinition` , `TocOptions` ) to named interfaces in `src/types/` .
- 

## APPENDICES A/B/C

### Finding A.1 — Appendix A ( `.agents/ symlink`) is stale

- **Location:** Appendix A
- **Description:** Documents a symlink that "no longer exists." The entry remains in `.gitignore` as template leftover.
- **Evidence:** "No longer exists at repo root."

- **Location:** Appendix A
- **Description:** Documents a symlink that "no longer exists." The entry remains in `.gitignore` as template leftover.
- **Evidence:** "No longer exists at repo root."
- **Impact:** Confusing for future agents — why document something that doesn't exist?
- **Severity:** Informational
- **Confidence:** Verified
- **Recommended fix:** Delete the appendix. Add a one-line note in §3 (" `.gitignore` contains template leftovers; ignore them") instead.

#### Finding B.1 — Appendix B (build output) is correct

- **Location:** Appendix B
- **Description:** Documents `npm run build → dist/index.html` via `vite-plugin-singlefile`.
- **Evidence:** Explicit.
- **Impact:** Positive.
- **Severity:** Informational (positive finding)
- **Confidence:** Reasoned
- **Recommended fix:** Carry forward; add the offline-build variant.

#### Finding C.1 — Appendix C (visual pipeline) duplicates §5.2

- **Location:** Appendix C
- **Description:** The visual pipeline diagram is essentially identical to §5.2's data flow.
- **Evidence:** Compare lines 927–939 and 1322–1334.
- **Impact:** Minor — redundancy without contradiction.
- **Severity:** Informational
- **Confidence:** Verified
- **Recommended fix:** Drop Appendix C; keep §5.2.

### 1.3 Cross-Cutting Observations

1. **The skill is over-fit.** Every design decision serves the audit-report use case. The skill's own §1 calls this out as a feature ("anti-generic mandate"). For a generalized skill, this is the central obstacle.
2. **The evidence contract is the skill's best idea.** Verified/Reasoned/Assumed/Unverifiable tags on every finding (and the corresponding badge system) are a transferable pattern that should be preserved verbatim in v2.0.0.
3. **Code-first theming via Tailwind v4 `@theme` is the right call.** No `tailwind.config.js`, no JS/TS theme objects — just CSS custom properties. This is the modern Tailwind idiom and should be preserved.
4. **The accessibility posture is aspirational, not verified.** The skill claims WCAG AAA but self-documents multiple AAA failures (touch targets, badge contrast, reduced motion). The fix is either to (a) actually meet AAA, or (b) honestly claim AA with AAA-aspirational notes.



5. **No automated testing of any kind.** No unit tests, no axe, no Lighthouse, no CI. For a single-purpose internal skill this is survivable; for a generalized skill targeting multiple authors and documents, it is the highest-leverage gap to close.
6. **Single-file build with runtime font dependence is a half-promise.** `vite-plugin-singlefile` inlines JS/CSS but not `@import` ed fonts. The artifact is not actually self-contained. v2.0.0 must offer an offline build mode.
7. **Documentation quality is excellent.** Section-by-section structure, anti-pattern tables, debugging guide, lessons-learned, pitfalls, best practices — this is the right shape for a skill. v2.0.0 should preserve the structure.

## 1.4 Reuse Value Assessment

| Module                                                        | Reuse in v2.0.0 | Action                                                      |
|---------------------------------------------------------------|-----------------|-------------------------------------------------------------|
| <code>@theme</code> token approach                            | High            | Generalize: templates provide their own <code>@theme</code> |
| <code>enhance.ts</code> preprocessor pattern                  | High            | Generalize: extend regex to any registered tag              |
| <code>toc.ts</code> slugger-sharing pattern                   | High            | Preserve; add unit test for slug parity                     |
| <code>MarkdownReport.tsx</code> components map                | High            | Preserve; allow template override                           |
| <code>StatusBadge</code>                                      | Medium          | Generalize: replace fixed keys with tag registry            |
| Evidence contract<br>(Verified/Reasoned/Assumed/Unverifiable) | High            | Preserve verbatim                                           |
| Severity token palette                                        | Medium          | Replace with generic accent scale                           |
| <code>comparative-analysis.md</code> content                  | None            | Move to example/fixture                                     |
| Anti-generic mandate                                          | Low             | Reframe as per-template, not global                         |
| <code>cn.ts</code> dead code                                  | None            | Delete                                                      |
| Pre-ship gate ( <code>tsc</code> + <code>build</code> )       | Low             | Expand with lint/test/a11y                                  |
| Appendix A ( <code>.agents/</code> )                          | None            | Delete                                                      |

## Part 2 — `markdown-to-web` v2.0.0 Skill Specification

```
---
name: markdown-to-web
description: >
  Renders an arbitrary Markdown document as a polished, single-file,
  accessible web page. Accepts any .md file plus an optional template
  (editorial long-form / technical docs / minimal print) and an optional
  tag registry (severity, confidence, status, custom). Produces a
  self-contained dist/index.html with WCAG 2.2 AA + AAA-aspirational
  accessibility, code-first theming, slug-parity navigation, and an
  evidence-tag badge system. Built on React 19 + Vite 7 + Tailwind v4
  + react-markdown. Use when the user asks to "render this markdown
  as a web page", "convert .md to HTML", "publish this document as a
  site", or "make a polished web version of this README/report/spec".
version: 2.0.0
tags:
  - react
  - vite
  - tailwindcss
  - markdown
  - html
  - single-file-build
  - accessibility
  - documentation
---
```

### 2.1 Identity & Design Philosophy

**One-sentence description:** A generalized, template-driven React application that renders any Markdown document as a polished, single-file, accessible web page — preserving the author's content as the single source of truth while applying an opinionated editorial design system.

**Design thesis:** *Content is data; rendering is configuration.* The Markdown file is the input. The template chooses the look. The tag registry chooses which inline annotations become badges. The build produces one self-contained `dist/index.html` that runs anywhere a browser can open a file — online or offline.

**Non-negotiable design rules:**

1. **Content is invariant.** Editing the Markdown never requires code changes. Adding a heading, table, code block, or `**Tag:** value` annotation is a content change, not a UI change.
2. **Templates are swappable.** Three ship in-box (editorial, technical, minimal). Each provides its own `@theme` tokens, layout, and component map. The user picks at invocation; the build wires it.
3. **Tags are registered, not hardcoded.** A document can use any `**<Tag>:** <value>` bullet as a badge as long as `<Tag>` is in the registry. Templates ship default registries; documents can extend.
4. **Single-file portability is real.** The default build inlines JS, CSS, and (optionally) fonts. The artifact runs from `file://`, a USB stick, or a static host with no CDN dependency.

5. **Accessibility is verified, not claimed.** Pre-ship runs `axe` + Lighthouse. The headline conformance claim is "WCAG 2.2 AA; AAA where feasible, with documented exceptions."
6. **Evidence over assertion.** When the source document contains findings (e.g., an audit report), each finding carries an explicit confidence tag. The renderer never upgrades "Unverifiable" to "Verified."

**Explicitly rejected (carried over from v1.0.1, scoped per-template):**

- Purple gradients on white
- Predictable rounded-card grids with left-border accents
- Generic "Inter/Roboto + gray-50" neutrality
- Hero sections with centered H1 + paragraph + CTA button
- Any component that could be dropped into a different template without visual friction

These rejections apply **per template**. The minimal print template may legitimately use Inter + gray-50 neutrality; that is its design register.

## 2.2 When to Use / When Not To

**Use this skill when:**

- The user provides a Markdown file ( `.md` ) and asks for a "web version," "HTML rendering," "polished page," or "publishable site."
- The document is long-form (1,000–50,000 words) and benefits from a Table of Contents.
- The document contains structured annotations ( `**Severity:** critical` , `**Status:** done` ) that should render as visual badges.
- The artifact must run offline or from `file://` .
- Accessibility conformance (AA minimum, AAA aspirational) is a requirement.

**Do NOT use this skill when:**

- The user wants a full Next.js application with server-side rendering, API routes, or database. Use `fullstack-dev` instead.
- The user wants a slide deck / presentation. Use `pptx` instead.
- The user wants a PDF. Use `pdf` instead.
- The document is a code project README that needs interactive code execution. Use a code-sandbox skill instead.
- The document is shorter than ~500 words; a styled HTML page is overkill — render inline.

## 2.3 Inputs Contract

The skill accepts:

| Input          | Required                              | Format                                                                 | Notes                                                                  |
|----------------|---------------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------|
| Markdown file  | Yes                                   | <code>.md</code> , UTF-8                                               | GFM extensions supported: tables, strikethrough, task lists, autolinks |
| Template       | No (default: <code>editorial</code> ) | <code>editorial</code>   <code>technical</code>   <code>minimal</code> | See §2.8                                                               |
| Tag registry   | No (default: template's)              | JSON or TS module                                                      | See §2.9                                                               |
| Theme override | No                                    | Partial <code>@theme</code> tokens                                     | Merges with template's tokens                                          |
| Title          | No (default: first H1)                | String                                                                 | Used in <code>&lt;title&gt;</code> , header, OG tags                   |
| Author         | No                                    | String                                                                 | Used in metadata                                                       |
| Offline fonts  | No (default: <code>false</code> )     | Boolean                                                                | When <code>true</code> , inlines fonts as base64                       |

### Markdown features supported:

- Headings H1–H4 (TOC extracts H2–H4 by default; configurable)
- Paragraphs, bold, italic, strikethrough
- Inline code, fenced code blocks (with language class for syntax highlighting via `rehype-highlight` — opt-in)
- Blockquotes
- Ordered/unordered lists, task lists
- Tables (GFM)
- Images (local paths resolved relative to the markdown file; remote URLs as-is)
- Links (external links get `target="_blank" rel="noopener noreferrer"` automatically)
- Horizontal rules
- HTML inline (passed through; sanitized via `rehype-sanitize` opt-in)

### Markdown features NOT supported (out of scope):

- Footnotes ( `[^1]` ) — add via `remark-footnotes` if a template needs it
- Math ( `$...$` ) — add via `remark-math` + `rehype-katex` if a template needs it
- Mermaid code blocks — add via `rehype-mermaid` if a template needs it
- Front matter — parsed and used for title/author if present; otherwise ignored

## 2.4 Tech Stack & Pinned Versions

| Layer           | Technology                          | Version          | Critical Note                                              |
|-----------------|-------------------------------------|------------------|------------------------------------------------------------|
| Framework       | React                               | 19.2.6           | Strict TypeScript; functional components only              |
| Build           | Vite                                | 7.3.2            | vite-plugin-singlefile for one-file output                 |
| Styling         | Tailwind CSS                        | 4.1.17           | CSS-first @theme in src/index.css ; no tailwind.config.js  |
| Markdown        | react-markdown                      | 10.1.0           | remark-gfm + rehype-slug                                   |
| Heading anchors | rehype-slug                         | 6.0.0            | Must match github-slugger output                           |
| TOC extraction  | github-slugger                      | 2.0.0            | Slug parity test required (§2.10)                          |
| Icons           | lucide-react                        | 1.28.0           | Menu, X, ExternalLink, Sun, Moon, Search                   |
| Class util      | clsx + tailwind-merge               | 2.1.1 / 3.4.0    | cn() helper in src/utls/cn.ts — actually used              |
| Packaging       | vite-plugin-singlefile              | 2.3.0            | Inlines JS/CSS; fonts opt-in via --offline                 |
| Fonts (offline) | @fontsource-variable/source-serif-4 | 5.0.0            | Inlined as base64 when --offline                           |
| Fonts (offline) | @fontsource-variable/inter          | 5.0.0            | Inlined as base64 when --offline                           |
| Fonts (offline) | @fontsource/jetbrains-mono          | 5.0.0            | Inlined as base64 when --offline                           |
| Accessibility   | @axe-core/playwright                | 4.10.0           | Pre-ship a11y gate                                         |
| TypeScript      | typescript                          | 5.9.3            | strict: true , noUnusedLocals , noUnusedParameters         |
| Linter          | eslint                              | 9.x              | @eslint/js + typescript-eslint + eslint-plugin-react-hooks |
| Test            | vitest                              | 2.x              | Unit tests for enhance.ts , toc.ts , slug parity           |
| Node            | —                                   | ≥20.19 or ≥22.12 | Vite 7 requirement                                         |

**Verification command:** `npm ls --depth=0` — every row above must appear with the exact version. Run in pre-ship.

## 2.5 Project Skeleton

```
markdown-to-web/
├─ package.json
├─ package-lock.json
├─ vite.config.ts
├─ tsconfig.json
├─ eslint.config.js
├─ index.html
├─ src/
│   ├─ main.tsx                # Entry: StrictMode + createRoot
│   ├─ App.tsx                 # Layout, drawer/theme state, TOC derivation
│   ├─ index.css               # Tailwind v4 @import + @theme (template-provided)
│   └─ content/
│       └─ document.md        # The input markdown (?raw import)
│   └─ templates/
│       ├─ editorial/
│       │   ├─ theme.css      # @theme tokens for editorial
│       │   ├─ components.tsx # Component map overrides
│       │   └─ layout.tsx     # Layout shell (sidebar + drawer + hero)
│       └─ technical/
│           ├─ theme.css
│           ├─ components.tsx
│           └─ layout.tsx
│       └─ minimal/
│           ├─ theme.css
│           ├─ components.tsx
│           └─ layout.tsx
│   └─ components/
│       ├─ MarkdownReport.tsx # react-markdown renderer + default components map
│       ├─ TableOfContents.tsx # Recursive TOC (sidebar + drawer)
│       ├─ Badge.tsx          # Tag-aware badge (replaces StatusBadge)
│       ├─ SkipLink.tsx       # Accessible skip-to-content
│       └─ ThemeToggle.tsx    # Light/dark/system toggle
│   └─ lib/
│       ├─ enhance.ts         # Tag-aware regex preprocessor
│       ├─ toc.ts             # H2-H4 outline extraction
│       ├─ tags.ts            # Tag registry loader
│       └─ slug-parity.test.ts # Unit test: github-sluggger vs rehype-slug
│   └─ utils/
│       └─ cn.ts              # clsx + tailwind-merge
│   └─ types/
│       ├─ template.ts        # TemplateConfig, TemplateProps
│       ├─ tag.ts             # TagDefinition, TagRegistry
│       └─ toc.ts             # TocItem (level 2 | 3 | 4)
├─ scripts/
│   ├─ build-offline.mjs      # Offline-font build variant
│   └─ generate-color-ref.mjs # Auto-generates $color reference from @theme
└─ tests/
    ├─ enhance.test.ts        # Tag preprocessor unit tests
    ├─ toc.test.ts            # TOC extraction unit tests
    └─ axe.test.ts            # Playwright + axe end-to-end
```

**File responsibility rule:** One file, one responsibility. `MarkdownReport.tsx` renders; `Badge.tsx` styles tags; `tags.ts` loads the registry; `enhance.ts` preprocesses strings. No file mixes concerns.

## 2.6 Design System (Code-First)

Each template ships its own `theme.css` with a Tailwind v4 `@theme` block. The default (editorial) theme inherits v1.0.1's palette and adds dark variants.

Editorial template `@theme` (light + dark):



```

@import "tailwindcss";

@theme {
  /* Typography */
  --font-serif: "Source Serif 4", ui-serif, Georgia, serif;
  --font-sans: "Inter", ui-sans-serif, system-ui, sans-serif;
  --font-mono: "JetBrains Mono", ui-monospace, monospace;

  /* Light: ink scale */
  --color-ink-950: #0b1615;
  --color-ink-900: #0f1e1c;
  --color-ink-800: #16302c;
  --color-ink-700: #204640;

  /* Light: paper scale */
  --color-paper-50: #fbfaf7;
  --color-paper-100: #f4f2ec;
  --color-paper-200: #e9e5da;

  /* Accent (shared with dark) */
  --color-teal-600: #0e7c86;
  --color-teal-700: #0b626a;
  --color-moss-500: #6fa661;
  --color-moss-600: #588650;

  /* Generic 5-step accent scale (replaces fixed severity tokens) */
  --color-accent-1: #b3261e; /* was: critical */
  --color-accent-2: #b45309; /* was: high */
  --color-accent-3: #a16207; /* was: medium */
  --color-accent-4: #3f6212; /* was: low */
  --color-accent-5: #1d4ed8; /* was: info */
}

/* Dark mode token overrides */
@media (prefers-color-scheme: dark) {
  @theme {
    --color-ink-950: #f4f2ec; /* inverted: paper becomes ink */
    --color-ink-900: #fbfaf7;
    --color-ink-800: #e9e5da;
    --color-ink-700: #d6d0c0;
    --color-paper-50: #0b1615;
    --color-paper-100: #0f1e1c;
    --color-paper-200: #16302c;
    --color-teal-600: #2ba8b3; /* brighter for dark bg */
    --color-teal-700: #0e7c86;
  }
}

/* Manual override class (toggled by ThemeToggle) */
[data-theme="dark"] {
  /* Same overrides as @media (prefers-color-scheme: dark) */
}

/* Reduced motion (the v1.0.1 gap, now fixed) */
@media (prefers-reduced-motion: reduce) {
  html { scroll-behavior: auto; }
}

```

Typography hierarchy (editorial template):

| Role                | Font           | Weight | Size (mobile)                                 | Size (sm+)        | Color                |
|---------------------|----------------|--------|-----------------------------------------------|-------------------|----------------------|
| H1 (document title) | Source Serif 4 | 600    | text-3xl                                      | sm:text-4xl       | ink-900              |
| H2 (section)        | Source Serif 4 | 600    | text-2xl                                      | sm:text-[1.75rem] | ink-900              |
| H3 (subsection)     | Source Serif 4 | 600    | text-xl                                       | —                 | ink-800              |
| H4 (sub-subsection) | Source Serif 4 | 600    | text-lg                                       | —                 | ink-700              |
| Body                | Inter          | 400    | base (16px)                                   | —                 | stone-700            |
| Meta / labels       | JetBrains Mono | 500    | text-xs                                       | —                 | teal-700             |
| Badge text          | Inter          | 600    | text-sm (14px, was 12px — fixes AAA contrast) | —                 | per-tag token        |
| Code block          | JetBrains Mono | 400    | text-sm                                       | —                 | paper-100 on ink-900 |

Color reference is auto-generated. Run `node scripts/generate-color-ref.mjs` to emit a markdown table from `@theme` . This prevents the drift v1.0.1 risks.

2.7 Three Templates

TEMPLATE A — EDITORIAL LONG-FORM (DEFAULT)

Use for: Audit reports, essays, long-form journalism, comparative analyses, design critiques.

Layout:

- Sticky dark header ( `z-40` ) with title, theme toggle, and (mobile) menu trigger
- Desktop: left sidebar ( `w-64` , sticky, `top-24` ) with TOC; main content column ( `max-w-3xl` )
- Mobile: slide-in drawer ( `z-50` ) with TOC; full-width content
- Hero: title + subtitle + meta chips (author, date, reading time)
- Footer: source link, generated date

Visual register: Bespoke editorial — Source Serif 4 display, warm paper background, teal/moss accents. This is the v1.0.1 design, generalized.

Default tag registry: Severity ( `critical` / `high` / `medium` / `low` / `informational` ) + Confidence ( `verified` / `reasoned` / `assumed` / `unverifiable` ).

## TEMPLATE B — TECHNICAL DOCS

**Use for:** API references, technical specifications, developer guides, RFCs.

### Layout:

- Sticky light header with search box (cmd-K palette, optional)
- Three-column desktop: left nav ( `w-60` ), content ( `max-w-4xl` ), right "on this page" outline ( `w-48` , sticky)
- Mobile: drawer nav; content; inline "on this page" accordion at top
- No hero — jump straight to H1 + first paragraph
- Footer: edit-on-GitHub link, version

**Visual register:** Utilitarian — Inter throughout (display + body), cool gray background, blue accent. Code blocks are first-class (syntax-highlighted, copy button).

**Default tag registry:** Status ( `stable` / `experimental` / `deprecated` / `removed` ) + Visibility ( `public` / `internal` / `restricted` ).

## TEMPLATE C — MINIMAL PRINT

**Use for:** Manuscripts, legal documents, printable reports, archival content.

### Layout:

- Single column, `max-w-2xl` , centered
- No header, no sidebar, no drawer — just title + content + page footer
- Print CSS: page breaks before H2, `@page { size: A4; margin: 2cm }` , no color in print (black on white)
- Optional "Download PDF" button using `window.print()`

**Visual register:** Minimal — system serif/sans/mono (no web fonts), black on white, no accent colors except for badges.

**Default tag registry:** None (badges disabled by default; opt-in via front matter).

## 2.8 Generalized Badge / Annotation Protocol

The v1.0.1 badge system hardcoded 9 keys. v2.0.0 replaces this with a tag registry.

**Tag registry schema ( `src/types/tag.ts` ):**

```

export interface TagDefinition {
  /** The tag name as it appears in markdown, e.g. "Severity", "Status". */
  name: string;
  /** The allowed values, each mapped to an accent step and optional label override. */
  values: Record<string, {
    accent: 1 | 2 | 3 | 4 | 5;
    label?: string; // defaults to the value, capitalized
  }>;
}

export type TagRegistry = Record<string, TagDefinition>;

```

Default registry (editorial template, `templates/editorial/tags.json`):

```

{
  "Severity": {
    "name": "Severity",
    "values": {
      "critical": { "accent": 1 },
      "high": { "accent": 2 },
      "medium": { "accent": 3 },
      "low": { "accent": 4 },
      "informational": { "accent": 5 }
    }
  },
  "Confidence": {
    "name": "Confidence",
    "values": {
      "verified": { "accent": 1 },
      "reasoned": { "accent": 2 },
      "assumed": { "accent": 3 },
      "unverifiable": { "accent": 4 }
    }
  }
}

```

Preprocessor ( `src/lib/enhance.ts` ):

```

import type { TagRegistry } from "@types/tag";

const BULLET_RE = /^(\\s*[-*+]\\s+|\\s*\\d+\\.\\.\\s+)\\s*\\s*([\\^*]+):\\s*\\s*\\s+(.+)$/gm;

export function enhanceMarkdown(
  markdown: string,
  registry: TagRegistry,
): { enhanced: string; warnings: string[] } {
  const warnings: string[] = [];
  const tagNames = Object.keys(registry).join("|");

  const enhanced = markdown.replace(
    BULLET_RE,
    (match, bullet: string, tag: string, value: string) => {
      if (!registry[tag]) {
        // Not a registered tag — leave unchanged, but warn if it looks like one
        if (/^(Severity|Confidence|Status|Visibility)$/i.test(tag)) {
          warnings.push(
            `Line contains "${tag}:" but "${tag}" is not in the registry. ` +
            `Add it to tags.json or rename the bullet.`,
          );
        }
      }
      return match;
    }
  );
  const v = value.trim();
  if (!registry[tag].values[v.toLowerCase()]) {
    warnings.push(
      `Unknown value "${v}" for tag "${tag}". ` +
      `Allowed: ${Object.keys(registry[tag].values).join(", ")}`,
    );
  }
  return `${bullet}**${tag}:** \\`$${v}\\`\\`;
},
);

return { enhanced, warnings };
}

```

### Improvements over v1.0.1:

1. Accepts all bullet styles ( `-` , `*` , `+` , ordered `1.` ) — fixes Finding 7.2.
2. Emits build-time warnings for unknown tags/values — fixes Finding 7.2.
3. Tag set is data, not code — fixes Finding 7.1.
4. Accent step (1–5) replaces hardcoded color tokens — fixes Finding 4.2.

**Badge component ( `src/components/Badge.tsx` ):**

```

import { cn } from "@utils/cn";

const ACCENT_STYLES: Record<number, string> = {
  1: "bg-red-50    ring-red-200    text-accent-1",
  2: "bg-amber-50  ring-amber-200   text-accent-2",
  3: "bg-yellow-50 ring-yellow-200  text-accent-3",
  4: "bg-lime-50   ring-lime-200   text-accent-4",
  5: "bg-blue-50   ring-blue-200   text-accent-5",
};

interface BadgeProps {
  tag: string;
  value: string;
  accent: 1 | 2 | 3 | 4 | 5;
}

export function Badge({ tag, value, accent }: BadgeProps) {
  return (
    <span
      className={cn(
        "inline-flex items-center rounded-full px-2.5 py-0.5",
        "text-sm font-semibold tracking-wide uppercase", // 14px, was 12px – fixes AAA
        "ring-1 ring-inset",
        ACCENT_STYLES[accent],
      )}
      data-tag={tag}
      data-value={value}
    >
      {value}
    </span>
  );
}

```

**Contrast fix:** Badge text is now `text-sm` (14 px) instead of `text-xs` (12 px). At 14 px, the AAA threshold relaxes to 4.5:1, which all accent-1 through accent-5 pairs clear. This addresses Finding 8.4.

## 2.9 TOC + Navigation Contract

TOC extraction ( `src/lib/toc.ts` ):

```

import GithubSlugger from "github-slugger";

export interface TocItem {
  level: 2 | 3 | 4;
  text: string;
  slug: string;
  children: TocItem[];
}

const HEADING_RE = /^(#{2,4})\s+(.)$/gm;

export function buildToc(markdown: string, maxDepth: 2 | 3 | 4 = 3): TocItem[] {
  const slugger = new GithubSlugger();
  const items: TocItem[] = [];
  const stack: TocItem[] = [];

  for (const match of markdown.matchAll(HEADING_RE)) {
    const level = match[1].length as 2 | 3 | 4;
    if (level > maxDepth) continue;
    const text = match[2].replace(/`/g, "").trim();
    const slug = slugger.slug(text);

    const item: TocItem = { level, text, slug, children: [] };

    while (stack.length && stack[stack.length - 1].level ≥ level) {
      stack.pop();
    }

    if (stack.length === 0) {
      items.push(item);
    } else {
      stack[stack.length - 1].children.push(item);
    }
    stack.push(item);
  }

  return items;
}

```

Slug parity test ( `src/lib/slug-parity.test.ts` ):



```

import { describe, it, expect } from "vitest";
import GithubSlugger from "github-slugger";
import { slug } from "github-slugger";
import rehypeSlug from "rehype-slug";
import { unified } from "unified";
import remarkParse from "remark-parse";
import remarkRehype from "remark-rehype";

const FIXTURES = [
  "Simple Heading",
  "Heading with `code`",
  "Heading with emoji 🍌",
  "中文标题",
  "Repeated Heading", // github-slugger dedupes with -1, -2
  "Repeated Heading",
  "  Leading whitespace  ",
  "Trailing hash #",
  "CamelCase",
  "snake_case",
  "kebab-case",
];

describe("slug parity: github-slugger ≡ rehype-slug", () => {
  for (const text of FIXTURES) {
    it(`matches for "${text}"`, async () => {
      const gs = new GithubSlugger();
      const fromSlugger = gs.slug(text);

      const md = `## ${text}`;
      const tree = unified()
        .use(remarkParse)
        .use(remarkRehype)
        .use(rehypeSlug)
        .parse(md);
      const fromRehype = (await unified()
        .use(remarkParse)
        .use(remarkRehype)
        .use(rehypeSlug)
        .run(tree)).children.find(
        (n: any) => n.tagName ≡ "h2"
      )?.properties?.id;

      expect(fromSlugger).toBe(fromRehype);
    });
  }
});

```

This test addresses Finding 2.2 — slug parity is now verified, not assumed. Run in pre-commit and CI.

**Active-section highlighting (new, not in v1.0.1):**

```
// In App.tsx
const [activeSlug, setActiveSlug] = useState<string>("");

useEffect(() => {
  const observer = new IntersectionObserver(
    (entries) => {
      for (const entry of entries) {
        if (entry.isIntersecting) {
          setActiveSlug(entry.target.id);
        }
      }
    },
    { rootMargin: "-80px 0px -80% 0px" },
  );

  for (const item of toc) {
    const el = document.getElementById(item.slug);
    if (el) observer.observe(el);
  }

  return () => observer.disconnect();
}, [toc]);
```

Pass `activeSlug` to `TableOfContents` to highlight the current section. This is a usability upgrade absent in v1.0.1.

## 2.10 Accessibility (WCAG 2.2 AA + AAA Aspirational)

| Feature                    | Implementation                                                                                                                                                               | Verification                                                                                  |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Skip-to-content            | <code>&lt;a href="#content" class="sr-only focus:not-sr-only focus:z-50 ..." &gt;</code>                                                                                     | Manual: Tab on load → focus moves to skip link → Enter → focus moves to <code>#content</code> |
| Focus visible              | Global <code>:focus-visible { outline: 2px solid var(--color-teal-600); outline-offset: 2px; }</code>                                                                        | Manual: Tab through all interactive elements; axe check <code>color-contrast</code>           |
| Heading hierarchy          | H1 → H2 → H3 → H4; no skipped levels                                                                                                                                         | Lighthouse check <code>heading-order</code>                                                   |
| Anchor offset              | <code>scroll-mt-24</code> on all anchored headings                                                                                                                           | Manual: click TOC link; heading appears below sticky header, not under it                     |
| Reduced motion             | <code>@media (prefers-reduced-motion: reduce)</code> disables smooth scroll + animations                                                                                     | Manual: macOS "Reduce motion" setting; TOC click should jump, not animate                     |
| Touch targets              | All interactive elements $\geq 44 \times 44$ px ( <code>min-w-11 min-h-11</code> or <code>p-2.5 +icon</code> )                                                               | Manual: measure in DevTools; axe check <code>target-size</code>                               |
| ARIA labels                | <code>aria-label</code> on nav, drawer trigger/close, theme toggle; <code>aria-hidden="true"</code> on decorative icons                                                      | axe check <code>aria-valid-attr</code> , <code>button-name</code>                             |
| Semantic landmarks         | <code>&lt;header&gt;</code> , <code>&lt;main&gt;</code> , <code>&lt;aside&gt;</code> , <code>&lt;nav&gt;</code> , <code>&lt;article&gt;</code> , <code>&lt;footer&gt;</code> | axe check <code>region</code>                                                                 |
| Color contrast             | Body text $\geq 7:1$ (AAA); meta text $\geq 4.5:1$ ; badge text (14 px) $\geq 4.5:1$                                                                                         | Lighthouse check <code>color-contrast</code>                                                  |
| Color isn't sole indicator | Badges use text + background tint, not color alone                                                                                                                           | Manual: simulate deuteranopia in DevTools                                                     |
| Keyboard nav               | Full keyboard operability; no keyboard traps                                                                                                                                 | Manual: Tab/Shift+Tab through entire page                                                     |
| Language                   | <code>&lt;html lang="..." &gt;</code> set from markdown front matter or detected                                                                                             | axe check <code>html-has-lang</code>                                                          |

### Pre-ship a11y command:

```
# Runs axe against the dev server (or built dist)
npm run a11y
# Equivalent to:
# playwright test tests/axe.test.ts
```

**tests/axe.test.ts :**

```
import { test, expect } from "@playwright/test";
import AxeBuilder from "@axe-core/playwright";

test("document passes WCAG 2.2 AA", async ({ page }) => {
  await page.goto("http://localhost:4173/"); # vite preview port
  const results = await new AxeBuilder({ page })
    .withTags(["wcag2a", "wcag2aa", "wcag22aa"])
    .analyze();
  expect(results.violations).toEqual([]);
});

test("document passes WCAG 2.2 AAA where feasible", async ({ page }) => {
  await page.goto("http://localhost:4173/");
  const results = await new AxeBuilder({ page })
    .withTags(["wcag2aaa", "wcag22aaa"])
    .analyze();
  // AAA violations are warnings, not failures, except for:
  // - target-size (touch targets)
  // - color-contrast (text contrast)
  const critical = results.violations.filter(
    v => ["target-size", "color-contrast"].includes(v.id),
  );
  expect(critical).toEqual([]);
});
```

This addresses Findings 8.1, 8.2, 8.3, 8.4.

## 2.11 Build & Deploy Recipes

### RECIPE A — DEFAULT SINGLE-FILE BUILD (CDN FONTS)

```
npm run build
# Output: dist/index.html (JS/CSS inlined; fonts load from Google Fonts CDN)
# Size: ~250-400 KB
# Deploy: any static host (GitHub Pages, Netlify, Vercel, S3, nginx)
```

**vite.config.ts :**

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";
import { viteSingleFile } from "vite-plugin-singlefile";
import { resolve } from "path";

export default defineConfig({
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: { alias: { "@": resolve(__dirname, "src") } },
  build: {
    target: "es2022",
    cssCodeSplit: false,
    rollupOptions: {
      output: { inlineDynamicImports: true },
    },
  },
});
```

## RECIPE B — OFFLINE SINGLE-FILE BUILD (FONTS INLINED)

```
npm run build:offline
# Runs: node scripts/build-offline.mjs
# Output: dist/index.html (JS/CSS/fonts all inlined as base64)
# Size: ~2-4 MB (depending on font subset)
# Deploy: any static host, USB stick, file://, air-gapped environment
```

### scripts/build-offline.mjs :

```
import { build } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

// The @fontsource packages ship font files in node_modules.
// Vite's `assetsInlineLimit` setting (set very high) inlines them as base64.

await build({
  plugins: [viteSingleFile()],
  build: {
    target: "es2022",
    cssCodeSplit: false,
    assetsInlineLimit: 100 * 1024 * 1024, // 100 MB — inline everything
    rollupOptions: { output: { inlineDynamicImports: true } },
  },
});
```

The `src/index.css` for offline mode replaces the Google Fonts `@import` with `@fontsource` imports:

```

/* Online (default) */
@import url("https://fonts.googleapis.com/css2?family=Source+Serif+4:opsz,wght@8..60,400;8..60

/* Offline (when --offline flag passed) */
/* @fontsource-variable/source-serif-4 */
/* @fontsource-variable/inter */
/* @fontsource/jetbrains-mono */
/* These are imported via JS in main.tsx, not CSS @import, so they get bundled */

```

In `main.tsx`:

```

// Conditional import based on env var set by build-offline.mjs
if (import.meta.env.VITE_OFFLINE_FONTS === "true") {
  await import("@fontsource-variable/source-serif-4");
  await import("@fontsource-variable/inter");
  await import("@fontsource/jetbrains-mono");
}

```

This addresses Finding 3.2.

## RECIPE C — GITHUB PAGES DEPLOYMENT

```

# 1. Set base in vite.config.ts
# base: "<repo-name>"

# 2. Build
npm run build

# 3. Deploy (using gh-pages or actions/upload-pages-artifact)
npx gh-pages -d dist

```

## RECIPE D — LOCAL FILE:// VIEWING

The default build works from `file://` because `vite-plugin-singlefile` removes all `<script type="module" src="...">` and `<link rel="stylesheet" href="...">` references — everything is inlined into one HTML file.

```

npm run build
open dist/index.html # macOS
# or: xdg-open dist/index.html # Linux
# or: start dist/index.html # Windows

```

## 2.12 Anti-Patterns & Pitfalls

| Area           | Don't                                                                                        | Do                                                                                                 |
|----------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Tags           | Hardcode tag keys in <code>Badge.tsx</code> ( <code>if (key === "critical")</code> )         | Use <code>TAG_REGISTRY</code> lookup; tags are data                                                |
| Slugs          | Manually set <code>id</code> on headings in Markdown                                         | Let <code>rehype-slug</code> derive; TOC matches via shared <code>github-slugger</code>            |
| Slugs          | Assume <code>github-slugger</code> and <code>rehype-slug</code> stay in sync across versions | Run <code>slug-parity.test.ts</code> in CI; pin both versions                                      |
| Fonts          | Assume <code>dist/index.html</code> works offline                                            | Use <code>build:offline</code> recipe; test by disconnecting network                               |
| Theming        | Add <code>tailwind.config.js</code> for new colors                                           | Extend <code>@theme</code> in <code>templates/&lt;name&gt;/theme.css</code>                        |
| Imports        | Use relative paths ( <code>../.. /components/X</code> )                                      | Use <code>@/components/X</code> alias                                                              |
| State          | Add global state (Context, Zustand) for document data                                        | Only client state is <code>drawerOpen</code> , <code>activeSlug</code> , <code>theme</code>        |
| A11y           | Claim "WCAG AAA" without verification                                                        | Run <code>npm run a11y</code> ; report actual violations                                           |
| Badges         | Use 12 px text for badges (fails AAA contrast)                                               | Use 14 px ( <code>text-sm</code> ) — clears AAA at 4.5:1                                           |
| Touch targets  | Use <code>p-1.5</code> (32 px) for drawer buttons                                            | Use <code>p-2.5</code> (44 px) minimum                                                             |
| Reduced motion | Set <code>scroll-behavior: smooth</code> without a reduce override                           | Always pair with <code>@media (prefers-reduced-motion: reduce)</code>                              |
| Build          | Run <code>npm run build</code> without <code>npm run typecheck</code>                        | Always run <code>typecheck &amp;&amp; lint &amp;&amp; test &amp;&amp; a11y &amp;&amp; build</code> |
| Content        | Edit component files to change document text                                                 | Edit <code>src/content/document.md</code> only                                                     |
| Templates      | Fork the whole project to switch templates                                                   | Pass <code>--template &lt;name&gt;</code> at invocation                                            |

## 2.13 Pre-Ship Checklist (Mandatory Verification Gate)

```
# 1. Typecheck (strict, noUnusedLocals/Parameters)
npm run typecheck
# Equivalent to: tsc --noEmit

# 2. Lint
npm run lint
# Equivalent to: eslint . --max-warnings 0

# 3. Unit tests (enhance, toc, slug parity)
npm run test
# Equivalent to: vitest run

# 4. Accessibility (axe + Lighthouse)
npm run ally
# Equivalent to: playwright test tests/axe.test.ts

# 5. Production build (single-file)
npm run build
# Or: npm run build:offline for the offline variant

# 6. Smoke test the build
npm run preview
# Open printed URL; verify:
#   - Header renders with title, theme toggle, (mobile) menu trigger
#   - Desktop sidebar + mobile drawer (resize < 1024 px)
#   - Full document renders with badges colored
#   - TOC links jump to correct sections
#   - Active section highlights in TOC
#   - Theme toggle switches light/dark
#   - Tab through page; focus rings visible on all interactive elements
#   - Open DevTools → Application → Lighthouse → Run; score ≥ 95 in all categories

# 7. Verify dependency versions
npm ls --depth=0
# Compare against §2.4 table; every version must match

# 8. Verify artifact is self-contained
# Online build: open dist/index.html with network → fonts load
# Offline build: open dist/index.html without network → fonts still render
```

**All eight gates must pass.** No gate may be skipped, weakened, or made non-blocking to ship.



## 2.14 Debugging Guide

| Symptom                                                         | Cause                                                               | Fix                                                                                                                                                  |
|-----------------------------------------------------------------|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Build fails with <code>vite-plugin-singlefile</code> error      | Plugin version mismatch or config                                   | Verify <code>vite.config.ts</code> has <code>viteSingleFile()</code> in plugins; <code>package.json</code> has <code>^2.3.0</code>                   |
| TOC anchor doesn't scroll                                       | Heading <code>id</code> missing or <code>scroll-mt-24</code> absent | Check <code>MarkdownReport.tsx</code> H2/H3/H4 components have <code>id={id}</code> and <code>scroll-mt-24</code> ; <code>rehype-slug</code> present |
| TOC anchor jumps to wrong heading                               | Slug parity broken (github-sluggger $\neq$ rehype-slug)             | Run <code>slug-parity.test.ts</code> ; pin both versions; never hand-edit slugs                                                                      |
| Badge shows wrong color                                         | Tag registry mismatch or unknown value                              | Check <code>enhance.ts</code> warnings output; verify <code>tags.json</code> has the tag and value                                                   |
| Badge renders as plain <code>&lt;code&gt;</code>                | Value not wrapped in backticks by <code>enhance.ts</code>           | Use exact bullet syntax <code>- **Tag:** value</code> ; ensure tag is in registry                                                                    |
| Heading missing from TOC                                        | Heading level $>$ <code>maxDepth</code> (default 3)                 | Increase <code>maxDepth</code> in <code>buildToc()</code> call, or restructure content                                                               |
| TypeScript error: unused local/param                            | Strict tsconfig                                                     | Delete or prefix with <code>_</code> ; run <code>npm run typecheck</code> after every edit                                                           |
| Dev server won't start                                          | Port 5173 occupied or Node $<$ 20.19                                | <code>lsof -i :5173</code> ; <code>node --version</code>                                                                                             |
| Fonts look wrong in <code>dist/index.html</code> (online build) | Network blocked; Google Fonts CDN unreachable                       | Use <code>npm run build:offline</code>                                                                                                               |
| Offline build is huge ( $>5$ MB)                                | Full font files inlined                                             | Subset fonts to only the glyphs used; use <code>fonttools</code> <code>pyftsubset</code>                                                             |
| Lighthouse a11y score $<$ 95                                    | Violations in axe output                                            | Run <code>npm run a11y</code> ; fix every violation; do not suppress                                                                                 |
| Theme toggle doesn't persist                                    | <code>localStorage</code> not wired                                 | Check <code>ThemeToggle.tsx</code> reads/writes <code>localStorage.theme</code>                                                                      |
| Active section doesn't highlight                                | <code>IntersectionObserver</code> not set up                        | Check <code>App.tsx</code> <code>useEffect</code> sets up observer for every TOC item's slug                                                         |
| <code>enhance.ts</code> warnings appear in build                | Unknown tag or value in markdown                                    | Add tag to <code>tags.json</code> or fix the markdown                                                                                                |

## 2.15 Extending the Skill

### ADDING A NEW TEMPLATE

1. Create `src/templates/<name>/` with `theme.css`, `components.tsx`, `layout.tsx`.
2. `theme.css` must define all tokens in §2.6 (light + dark variants).
3. `components.tsx` exports a partial components map that merges with defaults.
4. `layout.tsx` exports a React component receiving `{ title, toc, markdown, children }`.
5. Add the template name to the `Template` union type in `src/types/template.ts`.
6. Add a default `tags.json` if the template introduces new tag semantics.
7. Document the template in this skill file (§2.7).
8. Add a fixture document and an axe test for the new template.

### ADDING A NEW TAG

1. Add the tag to `tags.json` (or a document-local `tags.json`).
2. Define allowed values and accent steps (1–5).
3. Run `npm run test` — the `enhance.test.ts` suite should pick up the new tag automatically.
4. If the tag should appear in the TOC or header metadata, extend `layout.tsx` to extract it.

### ADDING A NEW MARKDOWN EXTENSION (FOOTNOTES, MATH, MERMAID)

1. Install the remark/rehype plugin: `npm install remark-footnotes`.
2. Add to `MarkdownReport.tsx` 's `remarkPlugins` array.
3. Add a component override in the components map for any new HTML element the plugin emits (e.g., `<sup>` for footnotes).
4. Add a fixture to `tests/` verifying the extension renders.
5. Document the opt-in flag in §2.3.

### ADDING SYNTAX HIGHLIGHTING

1. `npm install rehype-highlight`.
2. Add to `MarkdownReport.tsx` 's `rehypePlugins`.
3. Import a highlight.js CSS theme in `index.css` (or define one in `@theme`).
4. Add a "copy code" button component for `<pre>` blocks.

## 2.16 Verification & Evidence Contract

Every claim about the rendered output must carry an evidence tag. This contract is preserved verbatim from v1.0.1.

| Tag                 | Meaning                                   | When to use                                                                                               |
|---------------------|-------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Verified</b>     | Executed and observed directly            | After running <code>npm run a11y</code> , <code>npm run test</code> , or manual DevTools inspection       |
| <b>Reasoned</b>     | Logical inference from code, not executed | "Based on the <code>@theme</code> tokens, the contrast ratio is X:1" (without running a contrast checker) |
| <b>Assumed</b>      | Based on a stated assumption              | "Assuming the user opens the file in Chrome 120+"                                                         |
| <b>Unverifiable</b> | Environment does not allow verification   | "Mobile Safari behavior cannot be tested in this environment"                                             |

**Rule:** Never upgrade a tag. If a claim is Reasoned, do not present it as Verified. The skill's credibility depends on this honesty.

## Appendix A — Migration from `react-markdown-report` v1.0.1

| v1.0.1                                                                 | v2.0.0                                                                                            | Migration action                                                                                   |
|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>src/content/comparative-analysis.md</code>                       | <code>src/content/document.md</code>                                                              | Rename; the audit report becomes the editorial-template fixture                                    |
| <code>StatusBadge</code> with 9 hardcoded keys                         | <code>Badge</code> with tag registry                                                              | Replace component; move keys to <code>templates/editorial/tags.json</code>                         |
| <code>enhanceReportMarkdown</code> (regex on Severity/Confidence only) | <code>enhanceMarkdown</code> (regex on any registered tag)                                        | Replace function; warnings now emitted                                                             |
| <code>buildToc</code> (H2/H3 only)                                     | <code>buildToc</code> (H2-H4, configurable depth)                                                 | Replace function; pass <code>maxDepth: 3</code> for v1.0.1 parity                                  |
| <code>@theme</code> with severity tokens                               | <code>@theme</code> with accent-1–5 scale                                                         | Replace tokens; map old names to new in <code>tags.json</code>                                     |
| Google Fonts <code>@import</code> (online only)                        | <code>@import</code> (online) OR <code>@fontsource</code> (offline)                               | Conditional import in <code>main.tsx</code> based on <code>VITE_OFFLINE_FONTS</code>               |
| <code>html { scroll-behavior: smooth }</code> (no reduce guard)        | <code>+ @media (prefers-reduced-motion: reduce)</code>                                            | Add the media query                                                                                |
| Browser default focus outline                                          | Global <code>:focus-visible</code> style                                                          | Add the CSS rule                                                                                   |
| Touch targets 32–36 px                                                 | Touch targets $\geq 44$ px                                                                        | Update <code>App.tsx</code> button classes ( <code>p-1.5</code> $\rightarrow$ <code>p-2.5</code> ) |
| Badge text 12 px                                                       | Badge text 14 px                                                                                  | Update <code>Badge.tsx</code> ( <code>text-xs</code> $\rightarrow$ <code>text-sm</code> )          |
| Pre-ship: <code>tsc &amp;&amp; build</code>                            | Pre-ship: <code>typecheck &amp;&amp; lint &amp;&amp; test &amp;&amp; ally &amp;&amp; build</code> | Add npm scripts; install devDeps                                                                   |
| No tests                                                               | <code>vitest</code> + <code>@axe-core/playwright</code>                                           | Add test files in <code>tests/</code>                                                              |
| <code>cn.ts</code> dead code                                           | <code>cn.ts</code> used in <code>Badge.tsx</code> and template components                         | Wire <code>cn()</code> into class composition                                                      |
| Single template (editorial)                                            | Three templates (editorial/technical/minimal)                                                     | Extract editorial; add technical and minimal                                                       |

## Appendix B — Complete TypeScript Reference

## SRC/TYPES/TEMPLATE.TS

```
export type TemplateName = "editorial" | "technical" | "minimal";

export interface TemplateConfig {
  name: TemplateName;
  themeCss: string;          // path to theme.css
  components: Partial<ComponentsMap>;
  layout: React.FC<TemplateLayoutProps>;
  defaultTags: TagRegistry;
  tocMaxDepth: 2 | 3 | 4;
  offlineFonts: boolean;     // template-specific font strategy
}

export interface TemplateLayoutProps {
  title: string;
  subtitle?: string;
  author?: string;
  date?: string;
  readingTime?: string;
  toc: TocItem[];
  markdown: string;
  children: React.ReactNode;
}

export type ComponentsMap = {
  h1: React.FC<React.ComponentPropsWithoutRef<"h1">>;
  h2: React.FC<React.ComponentPropsWithoutRef<"h2">>;
  h3: React.FC<React.ComponentPropsWithoutRef<"h3">>;
  h4: React.FC<React.ComponentPropsWithoutRef<"h4">>;
  p: React.FC<React.ComponentPropsWithoutRef<"p">>;
  a: React.FC<React.ComponentPropsWithoutRef<"a">>;
  strong: React.FC<React.ComponentPropsWithoutRef<"strong">>;
  em: React.FC<React.ComponentPropsWithoutRef<"em">>;
  ul: React.FC<React.ComponentPropsWithoutRef<"ul">>;
  ol: React.FC<React.ComponentPropsWithoutRef<"ol">>;
  li: React.FC<React.ComponentPropsWithoutRef<"li">>;
  hr: React.FC<React.ComponentPropsWithoutRef<"hr">>;
  blockquote: React.FC<React.ComponentPropsWithoutRef<"blockquote">>;
  code: React.FC<React.ComponentPropsWithoutRef<"code">>;
  pre: React.FC<React.ComponentPropsWithoutRef<"pre">>;
  table: React.FC<React.ComponentPropsWithoutRef<"table">>;
  thead: React.FC<React.ComponentPropsWithoutRef<"thead">>;
  tbody: React.FC<React.ComponentPropsWithoutRef<"tbody">>;
  tr: React.FC<React.ComponentPropsWithoutRef<"tr">>;
  th: React.FC<React.ComponentPropsWithoutRef<"th">>;
  td: React.FC<React.ComponentPropsWithoutRef<"td">>;
};
```

#### SRC/TYPES/TAG.TS

```
export interface TagValueDefinition {
  accent: 1 | 2 | 3 | 4 | 5;
  label?: string;
}

export interface TagDefinition {
  name: string;
  values: Record<string, TagValueDefinition>;
}

export type TagRegistry = Record<string, TagDefinition>;
```

#### SRC/TYPES/TOC.TS

```
export interface TocItem {
  level: 2 | 3 | 4;
  text: string;
  slug: string;
  children: TocItem[];
}
```

### COMPONENT PROPS

- `MarkdownReport` : { `markdown`: string; `template`: TemplateConfig }
- `TableOfContents` : { `items`: TocItem[]; `activeSlug`?: string; `onNavigate`?: () => void }
- `Badge` : { `tag`: string; `value`: string; `accent`: 1 | 2 | 3 | 4 | 5 }
- `SkipLink` : { `targetId`: string }
- `ThemeToggle` : { `initialTheme`?: "light" | "dark" | "system" }

---

## Closing — Definition of Done & Verification Ledger

---

### What was verified

- **Verified (textually, from the skill document):** All Findings in Part 1 marked "Verified" — internal contradictions in the v1.0.1 skill text (WCAG AAA claim vs. 36 px touch targets; "single-file portability" vs. `font @import` runtime dependence; badge contrast self-report of 4.76:1 failing AAA).
- **Reasoned:** Findings marked "Reasoned" — logical inference from the skill's stated behavior, not re-executed in this environment.
- **Assumed:** v2.0.0 design recommendations assume the dependency versions in §2.4 are accurate at the time of skill installation. Run `npm ls --depth=0` to verify.

## What was NOT verified

- No project was bootstrapped; no `npm install`, `npm run build`, `npm run ally`, or `npm run test` was executed in this environment.
- The slug-parity test in §2.9 is written but not run; it requires `vitest` + `unified` + `remark-parse` + `remark-rehype` + `rehype-slug` + `github-slugger` installed.
- The `enhance.ts` regex in §2.8 is written but not tested against the full GFM fixture set.
- The `build-offline.mjs` script in §2.11 is a sketch; it requires testing with actual `@fontsource` packages to confirm fonts inline as base64.

## Commands the user can run to spot-verify

If the user wants to verify the v2.0.0 design before adopting it:

1. **Bootstrap a test project:** `npm create vite@latest markdown-to-web-test -- --template react-ts`
2. **Install deps:** `npm install react-markdown remark-gfm rehype-slug github-slugger lucide-react clsx tailwind-merge && npm install -D tailwindcss @tailwindcss/vite vite-plugin-singlefile vitest @axe-core/playwright`
3. **Copy the `enhance.ts`, `toc.ts`, `slug-parity.test.ts`, and `Badge.tsx` files from this skill into the test project.**
4. **Run:** `npx vitest run slug-parity.test.ts` — verifies the two slug algorithms produce identical output.
5. **Run:** `npx vitest run enhance.test.ts` — verifies the regex handles all bullet styles and emits warnings.
6. **Run:** `npm run build` with `vite-plugin-singlefile` — verifies the single-file artifact is produced.

## How to install the new skill

1. Save the Part 2 content (from the front matter through Appendix B) as `markdown-to-web_SKILL.md` in the skills directory.
2. Create a starter project at `skills/markdown-to-web/starter/` containing the file tree in §2.5 with minimal implementations of each file.
3. The skill is invoked when a user says "render this markdown as a web page" or "convert .md to HTML."
4. The agent reads the skill, copies the starter project, replaces `src/content/document.md` with the user's markdown, picks a template (or asks), and runs the pre-ship checklist.

## Confidence statement

This deliverable is **Reasoned** throughout for the v2.0.0 design — the design is internally consistent, addresses every High and Medium finding from the v1.0.1 review, and follows established React/Vite/Tailwind idioms. It is **not Verified** because no code was executed. The user should treat the v2.0.0 spec as a design document, not a tested implementation. The durable patterns (evidence

contract, slug parity test, tag registry, code-first theming) are high-confidence; the specific code snippets ( `enhance.ts` regex, `build-offline.mjs` sketch) are starting points that require runtime validation.